

CMPS 405 (Operating Systems)

Introduction

**This material is based on the operating system books by Silberschatz and Stallings*

CMPS 405 (Operating Systems)

What are you going to learn in this course?

Topic

- ✓ **Operating Systems theory and related algorithms**
- ✓ **Mastering Linux Operating System**
- ✓ **Batch scripting for Linux**
- ✓ **System programming with system calls and the C Library functions**
- ✓ **Simulations of CPU scheduling algorithms and other OS related algorithms**
- ✓ **Synchronization and Concurrency programming with applications in C and Java**
- ✓ **Network programming and application protocols development with Sockets in C and Java**
- ✓ **Undertake a project devoted for related OS topics**

What is an Operating System?

❖ Conclude it from these points:

- A program that acts as an intermediary between a user of a computer and the computer hardware.
- Resource allocator: manages and allocates resources.
- Control program: controls the execution of user programs and operations of I/O devices.
- Kernel: the one program running at all times (all else being application programs).

❖ Operating system goals:

- Execute user programs and make solving user problems easier.
- Make the computer system convenient to use.
- Use the computer hardware in an efficient manner.

Evolution of Operating Systems

- ❖ From **Mainframe** (batched, multi-programmed, time-sharing) to **microcomputers** to **desktops computers**, **multiprocessor systems** to **handheld computers** and **distributed systems**.

CPU Operation

- ❖ **Uni-processing:** one job at a time with no user interaction.

CPU Operation

❖ Batch Systems

- **Definition:** Early form of operating systems where jobs are collected, grouped (batched), and processed sequentially without user interaction during execution.
- **Key Features:**
 - **Non-interactive:** Users submit jobs (programs + data), and results are returned after execution.
 - **Sequential Execution:** Jobs are processed one at a time or in a predefined order.
 - **Efficient Resource Use:** Maximizes CPU utilization by minimizing idle time.
 - **Simple Scheduling:** Jobs are queued and executed in batches.
- **Advantages:**
 - Reduces CPU idle time by running jobs back-to-back.
 - Suitable for repetitive tasks (e.g., payroll processing, scientific computations).
- **Disadvantages:**
 - No real-time interaction or feedback for users.
- **Example Systems:** Early mainframes like IBM 1401 (1960s).

CPU Operation

❖ Multiprogramming:

- **Definition:** A technique that allows multiple processes to reside in memory simultaneously, ensuring the CPU is always executing one of them.
- **How It Works:**
 - When a process waits for an event (e.g., I/O completion), the operating system switches the CPU to another process.
 - This prevents the CPU from sitting idle and maximizes utilization.
- **Key Characteristics:**
 - Focuses on efficient resource utilization (CPU and I/O).
 - Designed for systems where user interaction is minimal (e.g., batch processing).
 - Requires memory management to handle multiple processes in memory.
 - Includes CPU scheduling to decide which process to run next.
- **Common Use Cases:** Early mainframes and batch processing systems.

CPU Operation

❖ Multitasking:

- **Definition:** An extension of multiprogramming where multiple processes are executed by rapidly switching between them, providing an **interactive** experience to users.
- **How It Works:**
 - Frequent context switching between processes creates the illusion that multiple tasks run simultaneously.
 - Processes execute for short bursts before switching, ensuring responsiveness.
 - Often involves interactive I/O, such as user inputs from a keyboard or mouse.
- **Key Characteristics:**
 - Designed for systems requiring constant user interaction (e.g., desktops, smartphones).
 - Requires mechanisms to manage response times and ensure fairness among processes.
- **Common Use Cases:** Personal computers and modern operating systems (Windows, macOS, Linux).

Parallel Systems (multiprocessor)

- ❖ Multiprocessor systems with more than one CPU in close communication.
- ❖ **Tightly coupled system** – processors **share memory** and **a clock**; communication usually takes place through the shared memory.
- ❖ **Advantages of parallel system:**
 - Increased throughput
 - Economical
 - Increased reliability
 - Graceful degradation: the ability to continue providing service proportional to the level of surviving hardware.
 - Fault tolerant system: is a system with graceful degradation.

Parallel Systems (multiprocessor)

❖ Symmetric multiprocessing (SMP)

- Each processor runs an identical copy of the operating system.
- All processors communicate with other processors by a shared memory.
- Many processes can run at once without performance deterioration.
- Most modern operating systems support SMP

❖ Asymmetric multiprocessing (AMP)

- Each processor is assigned a specific task; **master** processor schedules and allocates work to slave processors.
- No communication between processors as they are controlled by the **master** processor.
- More common in extremely large systems

Real-Time Systems

- ❖ Often used as a **control device** in a **dedicated application** such as controlling scientific experiments, medical imaging systems, industrial control systems, and some display systems.
- ❖ Well-defined fixed-time constraints resulting in Two types of real-time systems:
 - **Hard real-time system**
 - Considered to have **failed** if it does not complete its function within the deadline.
 - Secondary storage limited or absent, data stored in short-term memory, or read-only memory (ROM).
 - Examples of hard real-time systems: airplane sensor and autopilot systems.
 - **Soft real-time system**
 - System whose operation is degraded if results are not produced according to the specified timing requirement
 - Useful in applications (multimedia, virtual reality) requiring advanced operating-system features.

Distributed Systems

- ❖ **Distribute** the **computation** among **several physical processors**.
- ❖ A **computing environment** in which **various components** are **spread** across **multiple computers** over a **network**.
- ❖ **Loosely coupled system** – each processor has its **own local memory**; processors communicate with one another through various communications lines, such as high-speed buses or telephone lines.
- ❖ **Advantages of distributed systems.**
 - Resources Sharing
 - Computation speed up – load sharing
 - Reliability
 - Communications

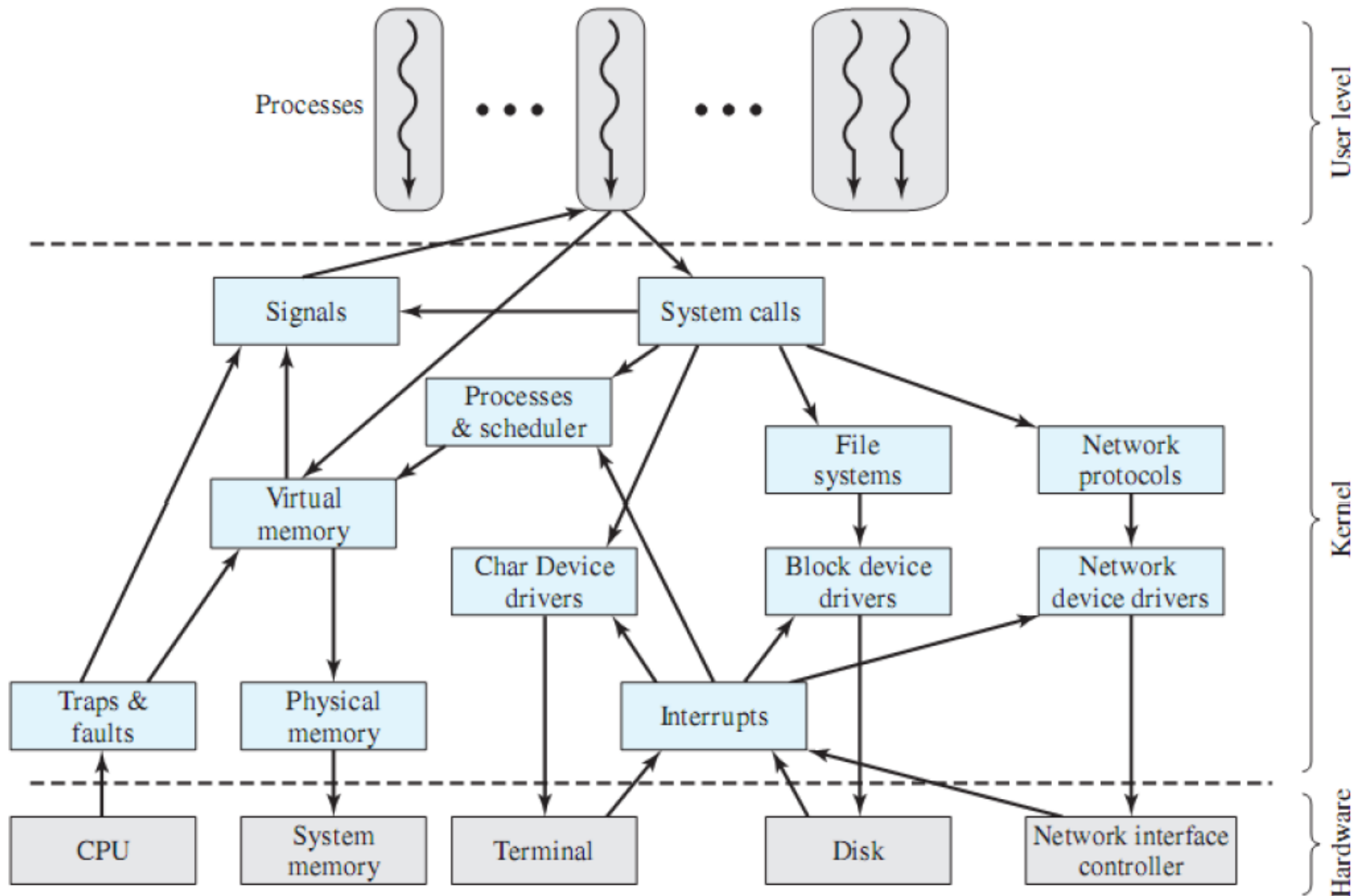
Operating Systems' Services

- ❖ User Interface
- ❖ Program Execution
- ❖ I/O Operations
- ❖ File-System Manipulation
- ❖ Communications
- ❖ Error Detection
- ❖ Resource Allocation
- ❖ Accounting
- ❖ Protection and Security

Operating systems design hierarchy

Level	Name	Objects	Example Operations
13	Shell	User programming environment	Statements in shell language
12	User processes	User processes	Quit, kill, suspend, resume
11	Directories	Directories	Create, destroy, attach, detach, search, list
10	Devices	External devices, such as printers, displays, and keyboards	Open, close, read, write
9	File system	Files	Create, destroy, open, close, read, write
8	Communications	Pipes	Create, destroy, open, close, read, write
7	Virtual memory	Segments, pages	Read, write, fetch
6	Local secondary store	Blocks of data, device channels	Read, write, allocate, free
5	Primitive processes	Primitive processes, semaphores, ready list	Suspend, resume, wait, signal
4	Interrupts	Interrupt-handling programs	Invoke, mask, unmask, retry
3	Procedures	Procedures, call stack, display	Mark stack, call, return
2	Instruction set	Evaluation stack, microprogram interpreter, scalar and array data	Load, store, add, subtract, branch
1	Electronic circuits	Registers, gates, buses, etc.	Clear, transfer, activate, complement

Linux Kernel Components



OS Structures

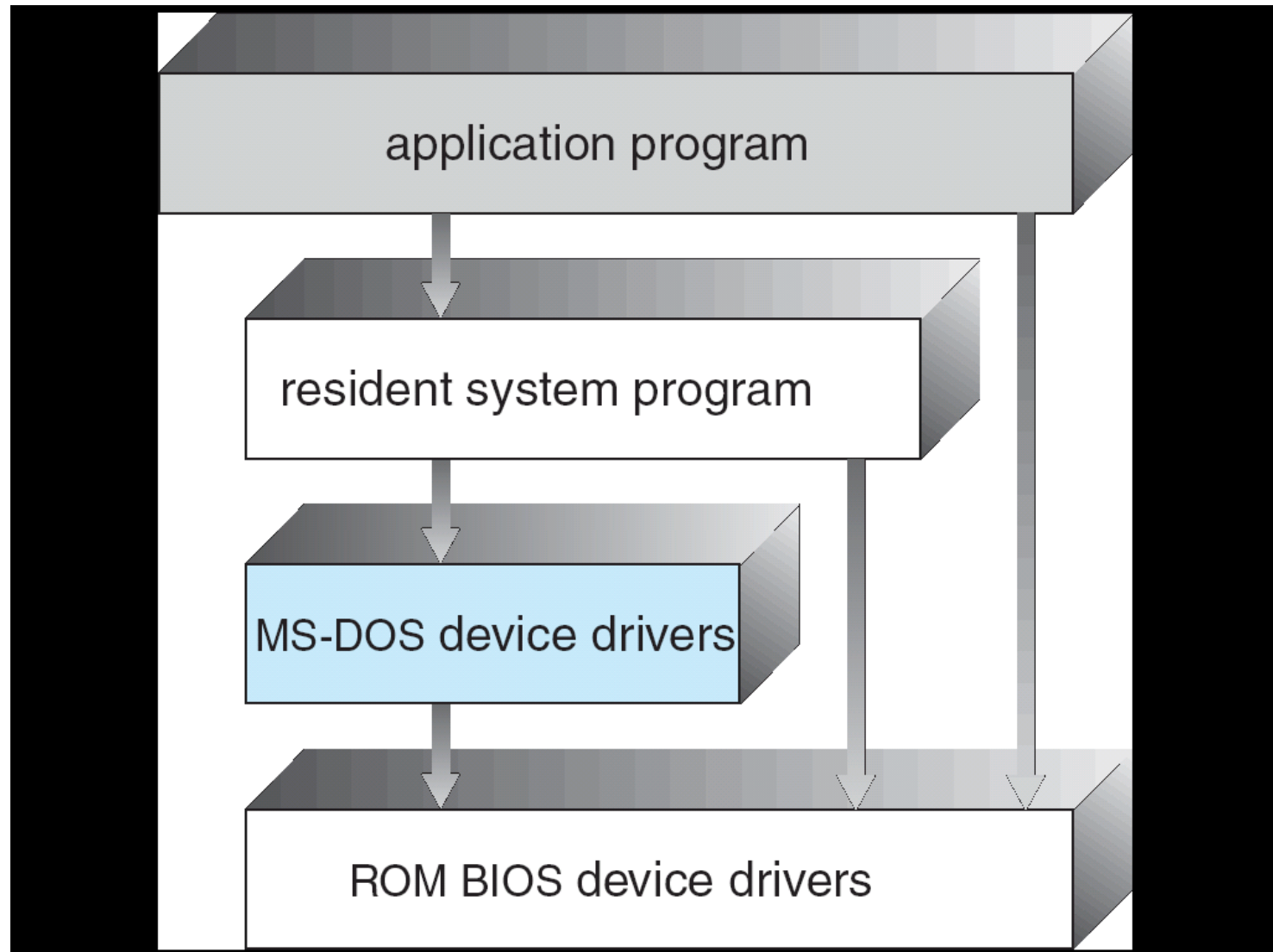
- ❖ Simple Structure
- ❖ Layered Approach
- ❖ Microkernel
- ❖ Monolithic Kernel
- ❖ Modular Kernel



OS Structures: Simple Structure

- ❖ Provides the **most functionality** in the **least space**: **limited hardware** it runs on.
- ❖ Does **not** have **well-defined structure**: interfaces and levels are not well separated
- ❖ The **base hardware** is **accessible** by user because the initial hardware does not provide dual mode or hardware protection.
 - The entire system crashes when user programs fail.
 - Such systems are difficult to implement and maintain.
- ❖ **Example: MS-DOS** where programs are able to access the basic I/O routines

Simple Structure

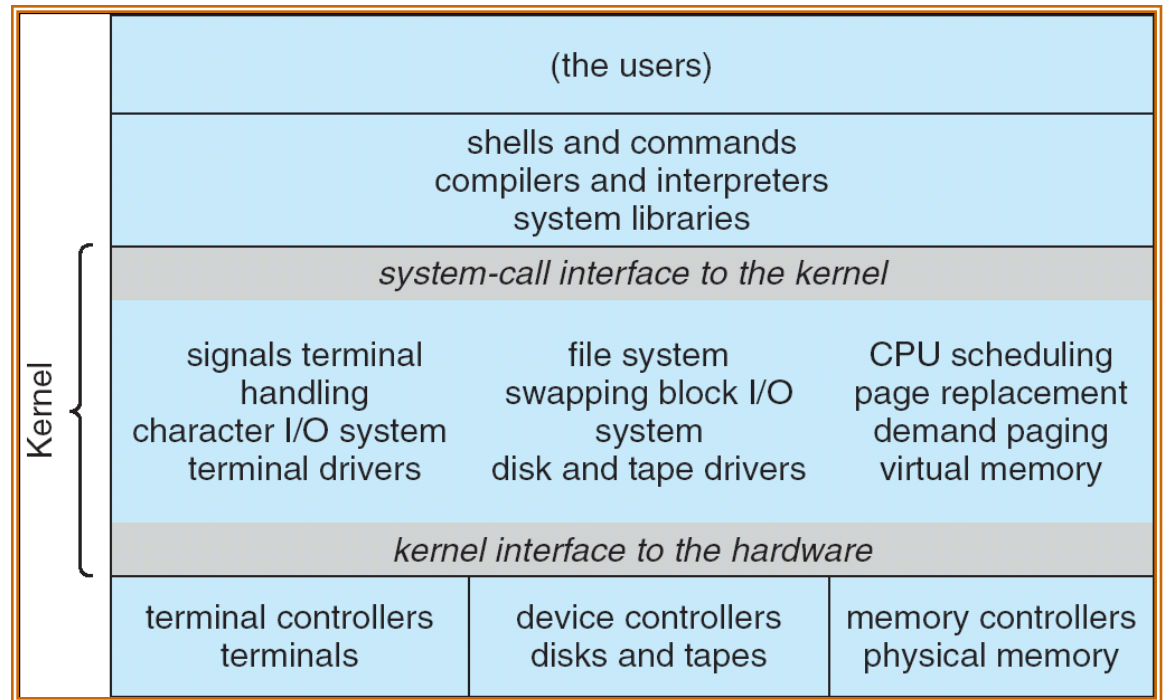
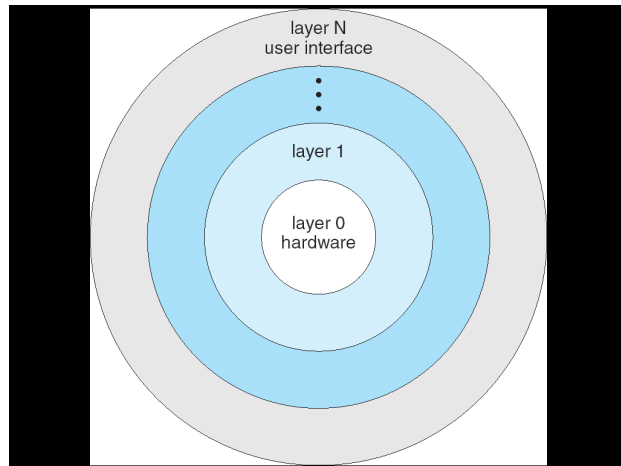




OS Structures: Layered Approach

- ❖ An **OS layer** is an implementation of an **abstract object** made up of **data** and their **operations**.
- ❖ In a **layered system**, **layer m** data and routines can be **invoked by layer m+1** on the other hand **layer m** can **invoke layer m-1** data and routines
- ❖ **Example: OS/2** which is a descendant of MS-DOS that adds **multitasking** and **dual-mode** operations as well as other features.

OS Structures: Layered Approach



OS Structures: Layered Approach

❖ Advantages:

- **Modularity:** resulted from system divided into layers with controlled interaction.
- Simplification of debugging and system verification.
- Direct user access to low-level facilities is not allowed

❖ Disadvantages:

- Defining appropriate layers data and operations.
- Less efficient than other types because of the nature of layered system requiring requests to be mapped into other requests down into layers.

OS Structures: Microkernel

- ❖ This method structures the operating system by **moving all nonessential components** from the **kernel** and implementing them as **system** and **user-level** programs.
- ❖ The **kernel is relatively small (microkernel)** providing **minimal process and memory management** as well as a **communication facility** using **message passing**.
- ❖ **Example:** Minix, Symbian



OS Structures: Microkernel

❖ Advantages:

- Ease of extending the operating system without modifications to the kernel.
- Easier to port from one hardware design to another.
- More secured and reliable: services run in user mode, any failure will not disturb the kernel.

❖ Disadvantages:

- Decrease of performance because of increased system overheads resulted from mapping system calls into messages to the appropriate user-level services for example.



OS Structures: Monolithic kernel

- ❖ An operating system architecture where the **entire operating system** is working in **kernel space**.
- ❖ File management, memory management, device management, and process management are **directly controlled** within the **kernel**.
- ❖ **Example:** OpenBSD, MS Windows (95, 98, ME)

OS Structures: Monolithic kernel

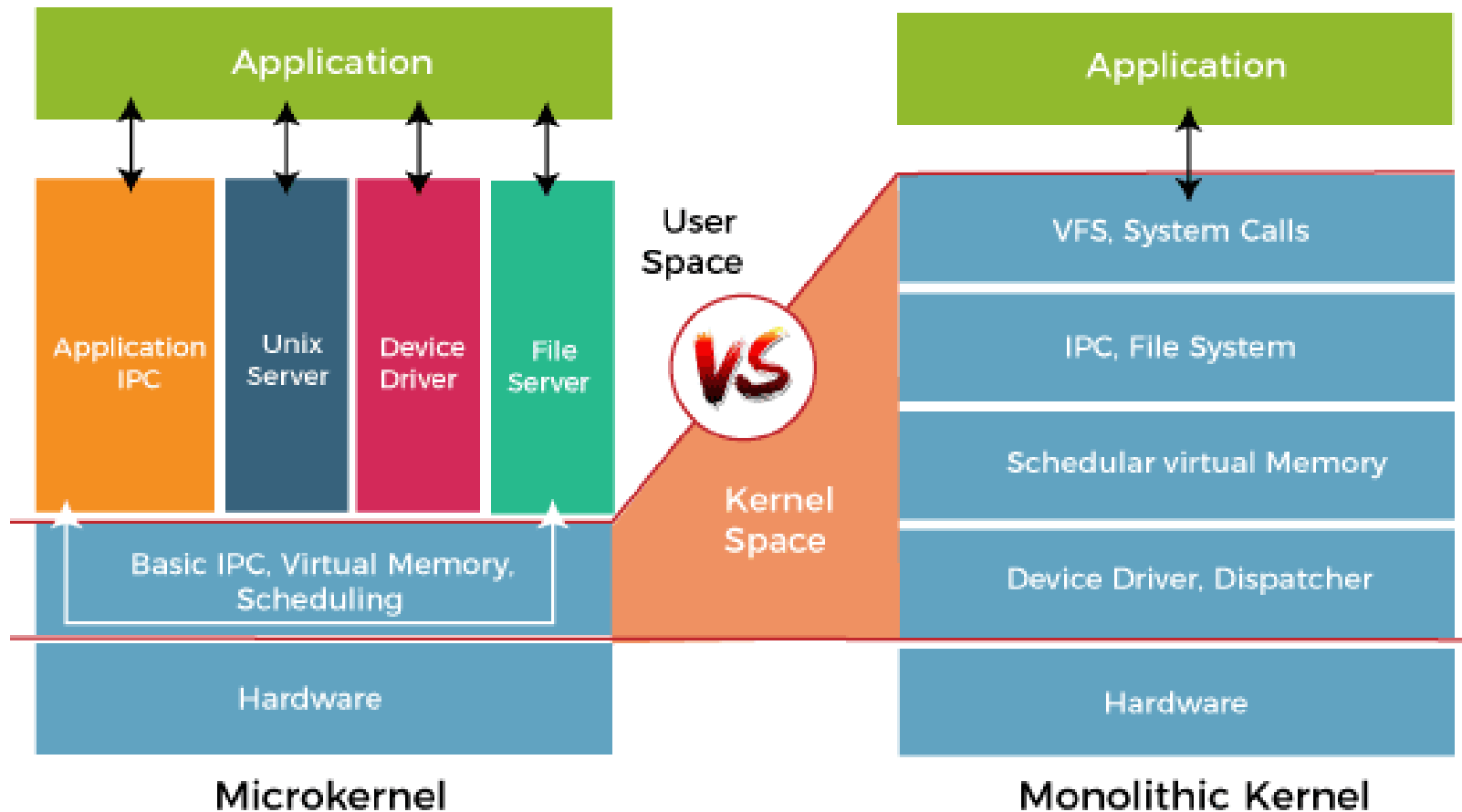
❖ Advantages:

- Simple and easy to implement structure.
- Faster execution due to direct access to all the services
- A process runs completely in single address space

❖ Disadvantages:

- The addition of new features or removal of obsolete features is very difficult.
- Security issues are always there because there is no isolation among various servers present in the kernel.
- If any service fails in the monolithic kernel, it leads to the failure of the entire system.

Microkernel vs. Monolithic kernel





OS Structures: Modular Kernel

❖ Definition:

- A **modular kernel** is an operating system architecture where the **kernel is designed to allow the loading and unloading of modules at runtime**, providing **flexibility** and **extensibility**.

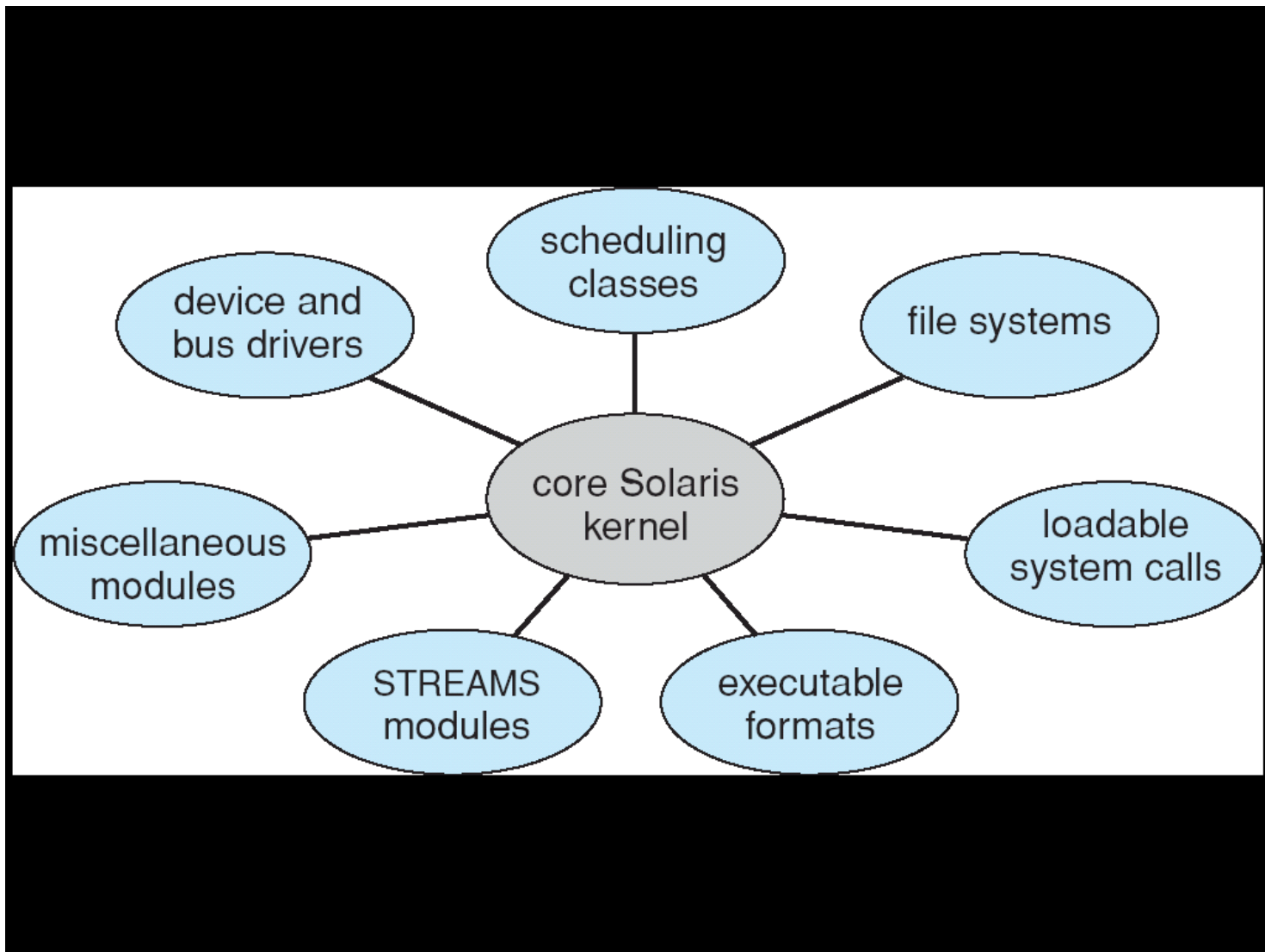
❖ Key Characteristics:

- **Core kernel** provides **essential services** (e.g., memory management, process scheduling).
- **Additional functionality** is added through **modules**, which are dynamically loaded or unloaded.
- Modules operate in **kernel space**, ensuring high performance.

❖ Structure Overview:

- **Core Kernel**: Handles core OS tasks.
- **Modules**: Add device drivers, file systems, or networking protocols without rebooting.

Modules Structure





OS Structures: Modular Kernel

❖ **Advantages:**

- Modules can be added or removed dynamically, adapting the OS to different hardware and use cases.
- Modules execute in kernel space, leading to faster operations compared to user-space implementations.
- Modular design isolates issues; a faulty module can be unloaded or replaced without requiring a system reboot.
- Bug fixes and updates can be applied to individual modules without affecting the core kernel.

❖ **Examples:** UNIX, Solaris, Linux and Mac OS X.



Implementations of OS

❖ **Old OS implemented in assembly while currently implemented in higher-level languages.**

- **Example: Linux and Win XP** are written mostly in **C** with some inserted **assembly code** for device drivers, registers and context switching.
- **Advantages:**
 - Fast implementation; compact; easier to understand and debug; easier to port (move to other hardware).
 - Better data structures and algorithms resulted in improving the performance of OSs.
- **Disadvantages:** reduce speed and increase storage requirements.

❖ **System performance must be monitored by:**

- Producing trace listing of system behavior (a file) to be used by analysis program to determine performance and find bottleneck routines and faults.
- Compute and display the performance measures in real time.

OS System Generation (SYSGEN)

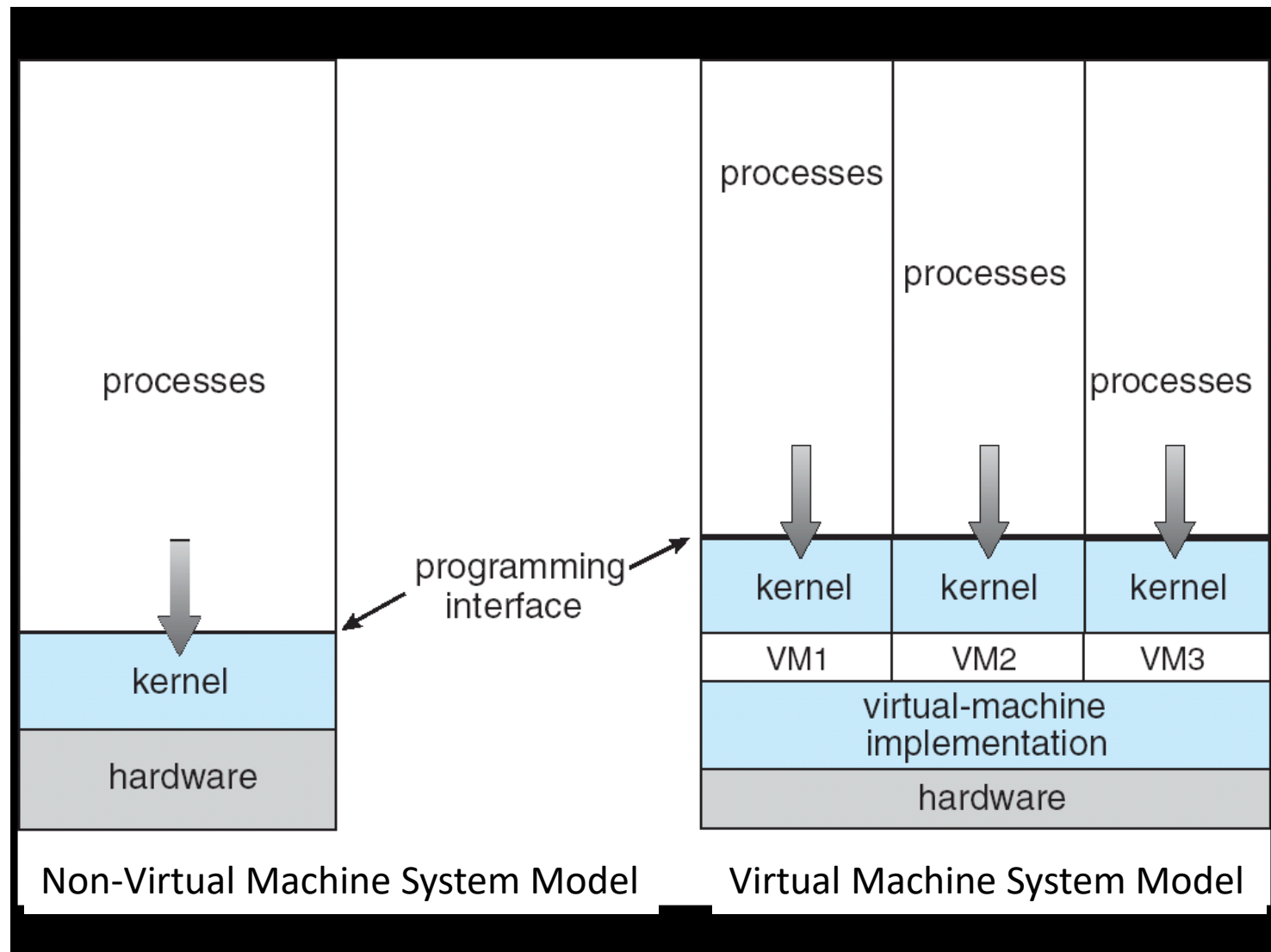
- ❖ It is the process of **configuring** and **generating operating system** to run on a **specific machine**. Implemented by a **SYSGEN** program that:
 - reads from a given file, or
 - asks the operator of the system for information concerning the specific configuration of the hardware system, or
 - probes the hardware directly to determine what components are there including information on CPU, memory, available devices and OS desired options.



Virtual Machines

- ❖ A **VM** is a **software-based emulation** of a **physical computer**. It allows multiple operating systems (OSs) to run on the same physical hardware, functioning as if they were separate, independent machines.
- ❖ Each virtual machine has its own virtual hardware, including CPU, memory, storage, network interface, and peripherals, provided and managed by the host system.
- ❖ **Key Features** of a **Virtual Machine** is **Isolation** where each VM operates independently, with its own operating system and applications, isolated from the host system and other VMs.

Virtual Machines



Virtual Machines

❖ Types of Virtual Machines

➤ **System Virtual Machines:**

- Provide a complete system environment.
- Allow the execution of an entire operating system.
- **Examples:** VMware Workstation, VirtualBox, KVM. They can allow the execution of one operating system on another, such as running Linux OS (guest system) on Windows (host system).

➤ **Process Virtual Machines:**

- Designed to run a single application or process, abstracting away the underlying operating system.
- **Examples:** Java Virtual Machine (JVM) for Java applications, .NET Common Language Runtime (CLR).

Virtual Machines

❖ Advantages of Virtual Machines

- **Resource Optimization:** Multiple VMs can run on a single physical machine, maximizing hardware utilization.
- **Flexibility:** VMs can run different operating systems on the same hardware.
- **Isolation and Security:** Issues in one VM do not affect others or the host system.
- **Testing and Development:** Ideal for testing software in isolated environments without risking the host system. For example, on a **macOS** laptop, a developer might use a system virtual machine like **VirtualBox** to run a **Windows 10 VM** for testing **Windows-specific applications**.

Virtual Machines

❖ Disadvantages of Virtual Machines

- **Performance Overhead:** Virtualization adds a layer of abstraction, which can reduce performance compared to running directly on hardware.
- **Resource Consumption:** Each VM requires its share of the host system's resources, which can limit the number of VMs on a single host.
- **Complexity:** Managing VMs adds complexity to system administration.

CMPS 405: OPERATING SYSTEMS

Command Interpreter, System Commands,
and Shell Scripting

This material is based on the operating system books by Silberschatz and Stallings

Contents:

- ❖ Command Interpreter (command line interface)
- ❖ System Commands
- ❖ Batch/Bash Script Files Programming in DOS and Linux

Command-interpreter

- ❖ A **command interpreter**, often referred to as a **shell**, is a program that provides a user interface for interacting with the operating system.
- ❖ It allows users to execute commands (by turning them into one or more **system calls**), manage files, run applications, and perform other tasks by typing instructions or using a graphical interface.
- ❖ It is usually **not part** of the **kernel** since the command interpreter is subject to changes.

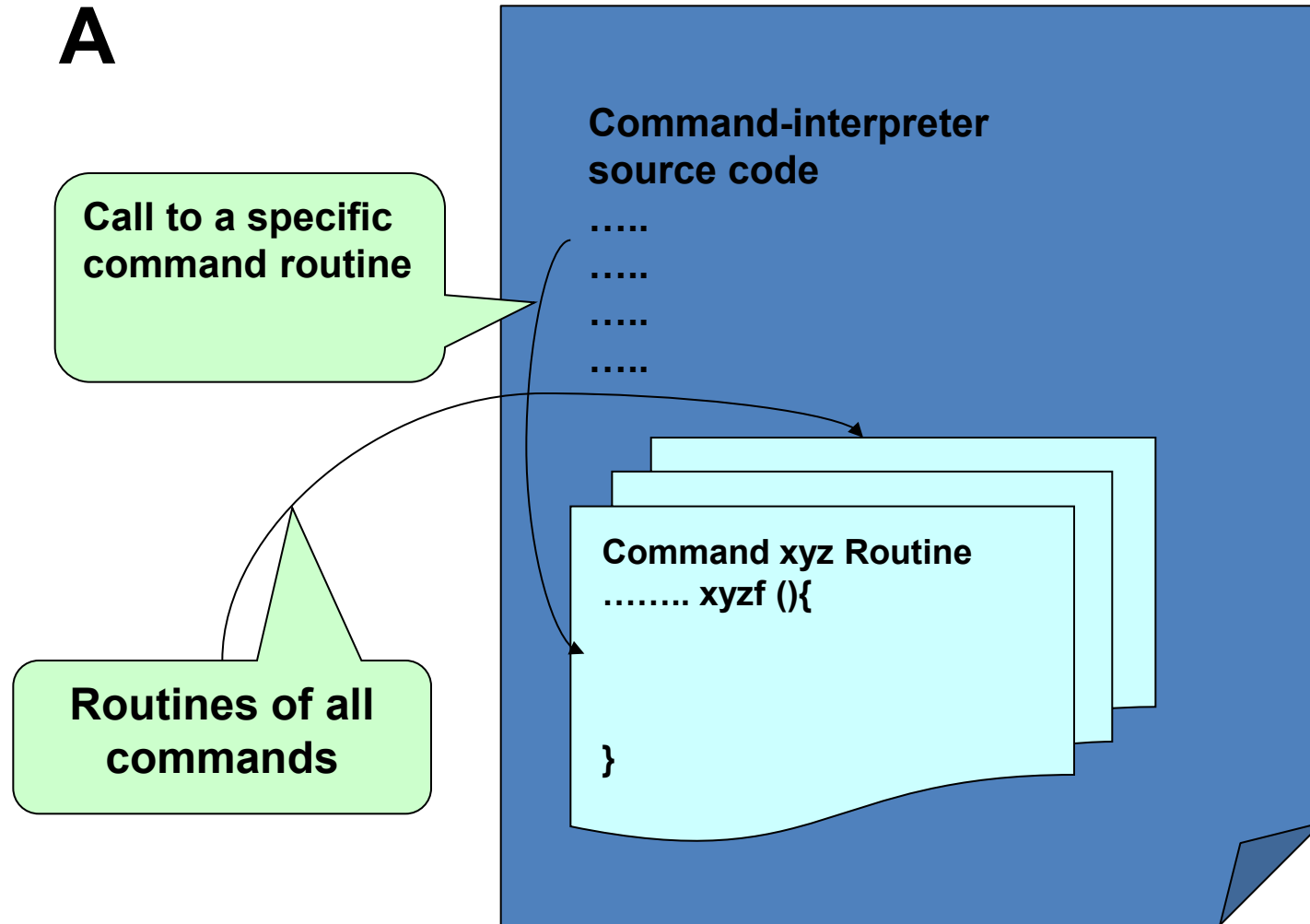
Implementations of Command-interpreter

❖ Approach A:

- ❖ The **command interpreter** itself contains the **code to execute the command**.
 - One **constrain** is the **size** of the **command interpreter** depends on the **number of supported commands**.
 - **No new commands can be added**. On the other hand it is **relatively faster**.

Implementations of Command-interpreter

A



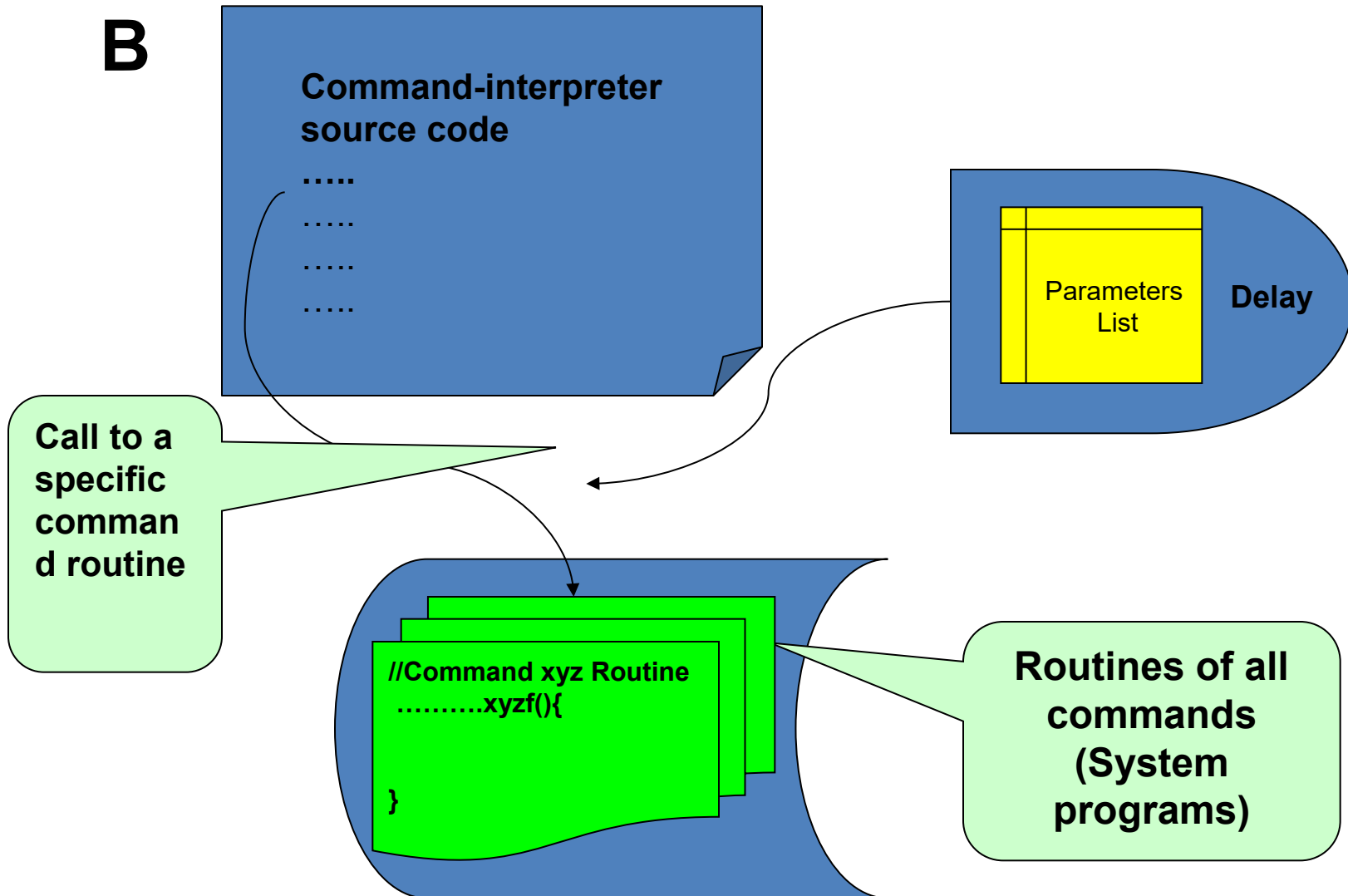
Implementations of Command-interpreter

❖ Approach B:

- ❖ **Most commands are implemented through system programs (UNIX). The command interpreter uses the command to identify a file to be loaded into memory and execute it.**
 - + The **command interpreter** is **small** and **flexible**.
 - + **New command** can be added by **creating files with proper names**. No need to have any changes on the command interpreter.
 - - On the other hand it **requires parameters passing** from the **command interpreter** to the **system program**.
 - - It is also **slower** because it **loads another program** and **executes it** rather jumping to a section of its code to execute.
 - - **Another problem** is the **inconsistency** of **parameters passed** to **system program** described by **programmers** of new system commands.

Implementations of Command-interpreter

B



Windows Batch file programming

- ❖ **Batch file programming** is used to **perform tasks from the command line**. The **script** is run as an **alternative to running** each command by hand.
- ❖ **How to:**
 - A **batch file** is actually a **text file** with **DOS commands** in it.
 - **Each DOS command** will be on a **separate line**.
 - Then, **instead** of a **.txt** extension, it has the extension of **.bat**.
 - You can **double click** the **batch file** or **type** its **name** at the **DOS prompt** to execute the commands.
 - To **create a batch file** in **Windows**, you need to **know** the **DOS commands**.
 - **Scripts** can **control** most **programs** and **system** which is important for **corporate admins** who usually use a **GUI tool** and **scripting**.

In class Demo with Windows

❖ Commands

- echo, copy, dir, cd, mkdir,

❖ Batch script programming

❖ Open **cmd** in windows, then do the following steps:

- cd ** ; go to root directory
- mkdir dummy** ; make directory named dummy
- cd dummy** ; go to dummy directory

- Create a batch file called **MyScript.bat** with the following contents:

```
echo This is one text file > File1.txt  
copy File1.txt File2.txt  
copy File2.txt File3.txt  
dir
```

- Run the batch file from the command line prompt:
MyScript.bat or **MyScript**

Linux Bash Shell Script programming

❖ How to:

- A **batch/bash file** is actually a **text file** with **commands** in it.
- Each command will be on a **separate line** or **separated** by **;**
- All **bash scripts** must tell the **OS** what to use as the **interpreter**.
The **first line** of any script should be **#!/bin/bash**
- Save it with the **extension** of **.sh**
- **Change** the **mode** of the **file** to be **executable**
chmod +x testbash.sh
- You can **double click** the **bash file** or **execute** it at the **command prompt** by typing
./testbash.sh
- The **script file** also **may not** have the **extension .sh**

In class Demo with Ubuntu

❖ Commands

- ls, touch, mkdir, cd, cp, chmod, ...

❖ Bash script programming

❖ Open terminal in ubuntu, then do the following steps:

cd ~ ; go to home directory

mkdir dummy ; make directory named dummy

cd dummy ; go to dummy directory

- Create a bash file called **testbash.sh** with the following contents:

#!/bin/bash

touch File1.txt

cp File1.txt File2.txt

ls

- Adjust mode of bash file to be executable:

chmod +x testbash.sh

- Run the batch file from the command line prompt:

./testbash.sh